全国大学生嵌入式芯片与系统设计竞赛 ’2026

FPGA 创新设计赛道（AMD）· 自主选题赛道（高级组）

设 计 报 告

RISC-V 图像识别加速系统

（端侧 INT8 CNN 异构软硬件协同加速系统）

目标平台： **PYNQ-Z2 / 征服者 7020（XC7Z020-1CLG400C）**

工具版本： **Vivado 2025.2、RISC-V GCC 10.1.0**

完成日期： **2026 年 11 月**

摘　要

面向端侧 AI 对 CNN 推理实时、低功耗、可观测、可复现的工程需求，本文基于蜂鸟 E203 轻量 RISC-V 内核，设计实现一套异构 CNN 加速系统。当前 RISC-V 端侧加速方案主要分为纯软件推理与独立 CNN 硬件 IP 两类：纯软件方案算力不足，独立 IP 方案往往缺少配套 CPU 调度、总线互联与性能观测体系。针对该工程缺口，本项目搭建 E203+ICB 总线桥 + 自研 INT8 CNN 外设的软硬件协同架构，配套完整驱动软件栈，实现 MNIST 推理端到端闭环，推理时延统计范围为首层卷积启动至末层全连接完成，推理端到端耗时 22ms，识别准确率达到 98.9%。本设计引入 PMU 与 CSR 双视角性能观测机制，定位数据搬运为主要系统开销，针对性完成权重 BRAM 预加载、Fast-IO 接口优化等，所有优化收益均通过板级实测量化验证。项目同步沉淀一键回归脚本、PMU 解析工具、寄存器模板，形成可复用的 RISC-V 端侧 CNN 加速工程套件，保障实验可复现，为同类异构加速开发提供工程参考。

关键词：RISC-V；蜂鸟 E203；INT8 量化卷积神经网络；性能监测单元；软硬件协同设计

目 录

摘 要 2
第 1 章 选题背景与问题定义 5
1.1 端侧 CNN 推理的四个现实约束 5
1.2 主要创新点 5
1.3 章节安排 5
第 2 章 系统结构与软硬件划分 7
2.1 总体架构 7
2.2 推理数据流 7
2.3 软硬件功能划分及依据 8
2.4 接口与地址映射 9
第 3 章 关键设计与实现 10
3.1 E203 底座 10
3.2 ICB → axi-lite 桥 10
3.3 INT8 CNN 内核 10
3.4 权重预加载与 WBASE 寻址 11
3.5 PMU 与 CPU 自定义 CSR 11
3.6 NICE 紧耦合控制器 12
3.7 软件栈与 MNIST 应用闭环 12
第 4 章 优化方法与实验设计 13
4.1 基于 CSR 与 PMU 的性能观测框架 13
4.1.1 第一轮观测：纯软件基线的乘法瓶颈 13
4.1.2 第二轮观测：总线传输与权重搬运瓶颈 14
4.1.3 第三轮优化：算子通路复用拓展 15
4.1.4 第四轮优化：紧耦合同步机制（NICE 指令扩展） 16
第 5 章 功能正确性验证 18
5.1 仿真回归 18
5.2 逐层数值比对 18
5.3 板级自检与批量统计 19
第 6 章 性能与资源结果 21
6.1 测试条件 21
6.2 版本对比 21

6.3 PMU 计数 22
6.4 时延分布与确定性 22
6.5 资源与时序 23
第 7 章 大模型协作与技能包 23
7.1 三个关键 LLM 交互记录 23
7.2 技能包 25
7.3 复用验证与红线 26
第 8 章 复现与开源 27
8.1 仓库结构 27
8.2 工具版本说明 27
8.3 从零复现的步骤 27
8.4 一键回归与开源声明 28
第 9 章 展望 28
9.1 优化方向 28
9.2 拓展方向 29
9.3 后续工作 29
附录 A 原始日志摘录 30
A.1 CoreMark（E203 底座） 30
A.2 纯 C 基线（V0） 30
A.3 加速器版本（V1 寄存器权重 / V2 ROM 权重） 30
A.4 最终版批量测试（V5，1000 张） 30
A.5 池化上板自检（含负数窗口） 30
A.6 Sobel 上板自检（K=3, OC=2） 31
附录 B 关键文件清单 32
参考文献 33

第 1 章　选题背景与问题定义

1.1　端侧 CNN 推理的四个现实约束

端侧 AI 在物联网嵌入式场景持续落地，对 CNN 推理提出实时低时延、低资源开销、性能可观测、实验可复现的需求。INT8 量化作为轻量化部署主流方案，能够在控制精度损失的前提下缩减模型体积，降低存储与带宽消耗。

在 RISC-V+CNN 异构加速方向上，已有两类主流实现路线：一类基于 E203 等轻量核纯软件推理，开发简单但算力受限；另一类独立实现 CNN 硬件加速器 IP，可获得较高计算吞吐，但大多聚焦计算核本身验证，较少配套完整 CPU 调度、总线互联与性能观测体系。现有研究更多关注卷积阵列本身的并行优化，在系统级瓶颈量化观测、可复现的对照实验框架、标准化可复用工程套件方面的工程沉淀相对偏少。

轻量级 RISC-V 内核蜂鸟 E203 仅依靠纯软件实现 CNN 推理时算力有限，难以满足图像识别实时性要求；单独开发 CNN 硬件加速器 IP，则缺失 CPU 任务调度、总线互联以及配套软件链路，无法构建完整可运行的嵌入式端到端系统。

在面向 RISC-V+CNN 异构加速的开发实践中，存在几个待解决难点：系统链路性能难以完整量化评估；硬件瓶颈定位缺少可靠的采集手段；优化措施效果缺少实测数据支撑；工程成果零散，实验复现与代码复用成本较高。

基于上述工程难点，本文凝练三项核心研究问题：构建 E203 内核与 CNN 加速器之间稳定可靠的 ICB 总线桥通信；搭建 PMU+CSR 双路性能观测体系，实现系统瓶颈精准定位；建立对照实验框架，量化评估各项优化策略带来的性能收益。

1.2　主要创新点

本作品只提炼三个创新点。

（1）系统级软硬件协同架构：把“开源 RISC-V 核 + 自研 ICB/axi-lite 桥 + INT8 CNN 外设 + 裸机软件栈 + MNIST 应用闭环”做成一条可上板、可复现的完整链路。整套链路在真实硬件上跑通，端到端数据可追溯到每一张图的周期数。

（2）PMU/CSR 双视角观测驱动的优化：先测后改。加速器侧 PMU 与 CPU 侧自定义 CSR 同时给出计数，再针对性地做权重 ROM 常驻，得到 AXI 写事务 −86.4%、推理链路端到端 −41.9% 的实测收益。实现“观测 → 定位 → 优化 → 复测”的闭环本身。

（3）可复用工程：ROM 一致性检查脚本、PMU 日志解析出表脚本、逐层黄金比对脚本与 LLM 协作记录。

1.3　章节安排

第 1 章为绪论，阐述选题背景、工程难点、研究问题与核心创新；

第 2 章介绍系统总体架构设计；

第 3 章描述硬件模块与 PMU 性能监测单元实现方案；

第 4 章开展基线测试与分层优化实验；

第 5 章完成 FPGA 上板功能验证与性能消融对比；

第 6 章分析 FPGA 资源占用与时序情况；

第 7 章介绍工程套件与可复用开发资源；

第 8 章总结项目成果，梳理开发过程中的技术难点；

第 9 章展望后续优化方向。

第 2 章　系统结构与软硬件划分

2.1　总体架构

系统由上位机、蜂鸟 E203 SoC、CNN 加速器外设三部分组成（图 1）。上位机通过 UART 以 115200 bps 承担批量测试的收发与统计；E203 SoC 内含 RV32I 核心、ITCM/DTCM、ICB 总线与 APB 外设；CNN 外设挂在与 ICB 相连的 axi-lite 从端口上，地址空间为 0x4000_0000 起的 256 字节。

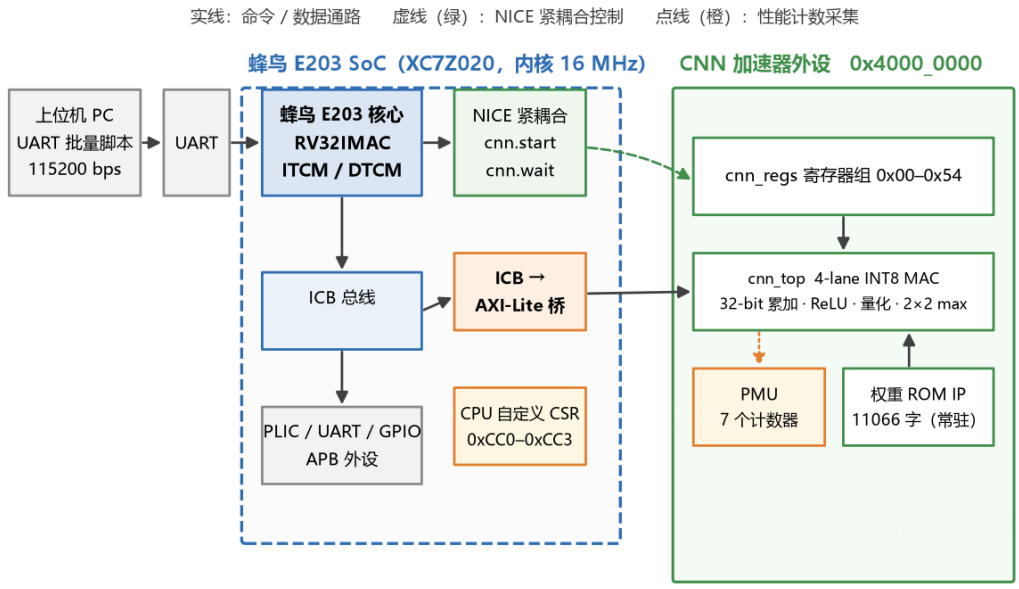


图 1　系统总体框图

图中三条通路需要区分：实线是命令与数据通路（CPU 经 ICB→axi-lite 读写 CNN 寄存器）；绿色虚线是 NICE 紧耦合控制通路（自定义指令直接触发加速器，不经过总线）；橙色点线是性能计数采集通路，加速器侧 PMU 与 CPU 侧 CSR 分别把内部事件累计成寄存器，供软件与脚本读取。

2.2　推理数据流

一次完整推理的数据流如图 2 所示。图像以 int8 写入 IDATA 寄存器，由硬件完成 C1 卷积、S2 池化、C3 卷积、S4 池化、C5 卷积、F6 全连接与 OUT 全连接，输出的 10 个 logit 由 CPU 顺序读回并做 argmax。所有卷积、池化与全连接层共用同一套 4-lane MAC 数据通路：F6 与 OUT 通过 H=W=1、K=1 的退化配置变成 1×1 卷积。

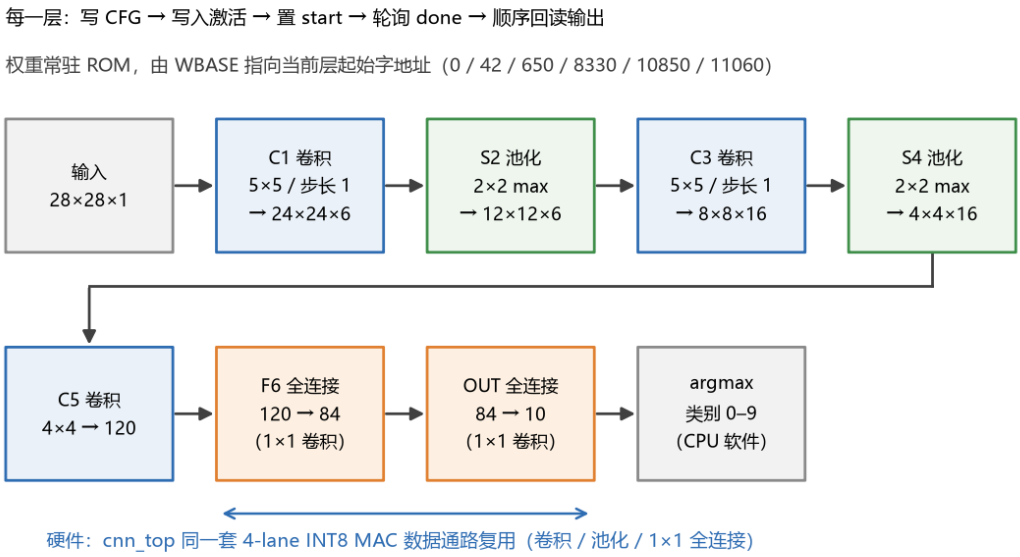


图 2　LeNet-5 INT8 推理数据流与软硬件划分

每层的软件流程是固定的五步：写 CFG0/CFG1 配置形状与量化参数、写 WBASE 指向该层权重在 ROM 中的起始字地址、写入激活、置 start 并等待 done、顺序回读输出。

2.3　软硬件功能划分及依据

软硬件划分的三个可判定的准则：算术密集且形状规整的进硬件；需要灵活判断或与外部交互的留软件；需要被观测的单独做成寄存器。表 1 是完整划分表。

表 1　软硬件功能划分及依据

功能	承担方	划分依据
卷积 / 池化 / 全连接	CNN 外设（硬件）	约 41.6 万次乘加且形状规整，适合 4-lane 并行；软件实现要 804 万周期
权重存储与寻址	ROM IP + WBASE 寄存器	权重总量 11066 个 32 位字，逐次经寄存器搬运会产生数万次 AXI 写事务
层配置、量化参数	软件写 CFG0/CFG1	参数随层变化，做成硬件参数会失去灵活性
激活搬运	软件	-
argmax 与结果判定	软件	-
批量测试与统计	上位机 Python 脚本	测试逻辑与硬件解耦，便于换数据集
加速器侧事件计数	PMU（6 个计数器）	-
CPU 侧指令/停顿计数	自定义 CSR（4 个计数器）	-
启动与等待控制	NICE 紧耦合接口	MMIO 轮询每次都付总线代价；紧耦合可把控制开销降到单指令

注：加速器侧 PMU 与 CPU 侧 CSR 的计数口径不同，不可直接相加，详见 4.1 节。

2.4　接口与地址映射

CNN 外设占用 0x4000_0000 起的 256 字节地址空间，其中 0x00–0x30 为功能寄存器（配置、输入、偏置、输出指针与版本），0x34–0x50 为 PMU 寄存器，0x54 为 WBASE。CPU 侧另扩展了 4 个只读 CSR，占用 0xCC0–0xCC3 这段非标准编码空间。

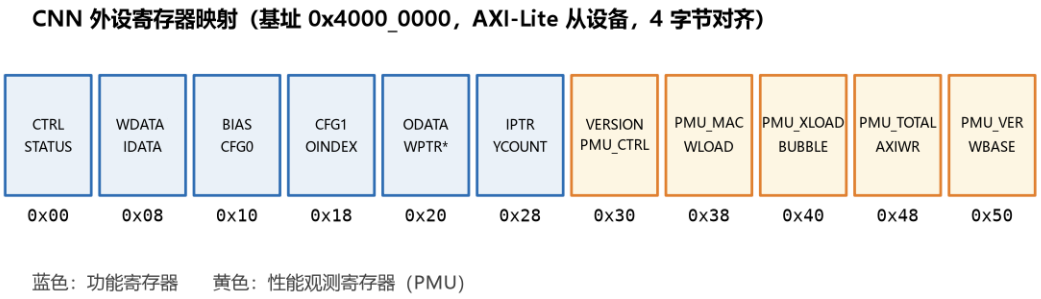


图 3　CNN 外设寄存器映射与 CPU 侧扩展 CSR

寄存器映射的完整定义见表 2。表中“保留”项（0x08 的 WDATA、0x24 的 WPTR）在权重改为 ROM 常驻后已无作用。

表 2　CNN 外设寄存器映射

偏移	名称	读写	说明
0x00	CTRL	W	[0] start　[2] 清输入指针　[3] 清偏置指针　[31] 选择 MMIO/NICE 启动模式
0x04	STATUS	R	[0] busy　[1] done
0x08	WDATA	W	保留（权重改为 ROM 常驻后不再使用）
0x0C	IDATA	W	写一次推进一个输入字节，地址由内部 iptr 自增
0x10	BIAS	W	写一次推进一个 int32 偏置，地址由内部 bptr 自增
0x14	CFG0	W	[7:0] H　[15:8] W　[23:16] C　[31:24] K
0x18	CFG1	W	[7:0] OC　[9:8] stride　[13:10] pad　[18:14] shift　[19] relu　[20] pool
0x1C	OINDEX	W	输出读地址
0x20	ODATA	R	符号扩展后的 int8 输出
0x24	WPTR	R	保留，读恒 0
0x28	IPTR	R	当前输入指针（调试用）
0x2C	YCOUNT	R	已产生的输出点数
0x30	VERSION	R	固定 0x0001_0000，驱动据此自检
0x34	PMU_CTRL	W/R	[0] 使能　[1] 写 1 清零全部计数器
0x38–0x4C	PMU_*	R	MAC / WLOAD / XLOAD / BUBBLE / TOTAL / AXIWR 六个计数
0x50	PMU_VERSION	R	固定 0x0001_0000
0x54	WBASE	W/R	当前层权重在 ROM 中的起始 32 位字地址
CSR 0xCC0–0xCC3	mulcnt / loadcnt / storecnt / stallcnt	R	CPU 侧指令与停顿计数，只在事件发生时自增

表 2 是 CNN 加速器外设的寄存器映射表，用来定义 E203 内核通过总线访问这个 CNN 硬件 IP 时，各个 MMIO 寄存器的地址、读写属性和功能，是软硬件交互的接口规范。

第 3 章　关键设计与实现

3.1　E203 底座

蜂鸟 E203 是一款两级流水、RV32IMAC 的开源 RISC-V 处理器，带 ITCM/DTCM、CLINT、PLIC 与 NICE 接口，采用 Apache-2.0 许可。本作品使用其 hbird_sdk 裸机环境，把 ELF 下载到 ITCM 中运行。

E203 的 CoreMark 实测在 ILM 下载模式下运行 500 次迭代，总 tick 236,054,487，耗时 14.81 s，得到 33.756 iterations/s；按实测主频 15,936,716 Hz 折算为 2.118 CoreMark/MHz（附录 A.1）。

3.2　ICB → axi-lite 桥

E203 的外部访问走的是 ICB 总线，加速器使用 axi-lite 从接口。ICB 是简单的 valid/ready 命令-响应两段式，axi-lite 有独立的 AW/W/B 与 AR/R 五通道。通过 ICB2axi-lite 桥把一次 ICB 命令拆成 axi-lite 的地址与数据两拍，再把 B/R 响应合并回一次 ICB 响应。

桥的验证分两级：先用 testbench 做“写寄存器→读回比对→连续读写不死锁”的功能测试，再用 CPU 侧真实指令驱动（CPU → ICB → axi-lite → cnn_regs → cnn_top）跑全通路仿真。

3.3　INT8 CNN 内核

cnn_top 是一个“形状可配置的一层”加速器，核心结构包括：4 路并行 MAC、32 位累加器、移位量化与饱和、权重 ROM 接口、输入/输出缓冲、以及一个四状态机（S_IDLE / S_MAC / S_QUANT / S_DONE）。它一次处理一个输出点的一轮 4 个 tap，累加完成后再统一做量化与 ReLU，因此同一个数据通路可以服务卷积、池化与 1×1 全连接三种形态。

几个关键设计决策值得说明。

选 4 路而不是 8 路，是因为 32 位 ROM 字宽正好装 4 个 int8，且 XC7Z020 上 4 路只占 11 个 DSP，留足余量给后续扩 lane。

累加器位宽取 32 位而非常见的 16 位。K=5、C=16 时最坏情况累加值为 $5\times5\times16\times127\times127 \approx 6.4\times10^6$，16 位必然溢出；本作品在整个设计期内没有出现过一次定点溢出，正是这条早期决策的价值。

权重按 tap 顺序打包进 32 位字（每字 4 个 int8，对应 4 个 lane），使一次 ROM 读就能喂满 4 个 MAC。ROM 字布局为 [OC][TAP_GROUP][LANE]，地址 = w_base + oc × num_tap_groups + tap_group。

ROM 是同步读 IP，输出比地址晚一拍。因此在进入 S_MAC 的第一拍用 mac_first 标志跳过，让 w_word 与 tcur 对齐。

量化统一在 cnn_quant 里做：先加偏置，再算术右移 shift 位，再按需 ReLU，最后饱和到 int8。把饱和集中在一处，使得“改量化”只需要改一个模块。

池化不新增乘法器，而是把累加改成取最大：acc 初值取 −128（int8 最小值），无效 lane 按 −128 参与比较，偏置在池化模式下强制置 0，避免“池化还叠偏置”的语义错误。全连接不单独

做乘法阵列，而是把 H=W=1, K=1 的退化配置交给同一套 MAC——此时 tap_next 只在通道维推进，等价于一次矩阵向量乘。

这样做的代价是核的通用性有限（不支持空洞卷积、不支持非对称 padding 组合），收益是资源极小：整个设计在 XC7Z020 上只用了 11 个 DSP 与 47.5 个 BRAM Tile（表 15）。对端侧 LeNet 级别的工作负载，这个取舍是划算的。

3.4 权重预加载与 WBASE 寻址

最初的版本里，权重和偏置一样，由 CPU 一个字节一个字节写进加速器（WDATA 寄存器）。PMU 测量数据暴露了问题：PMU_AXIWR（axi-lite 写事务数）高达 52,161，PMU_WLOAD（权重装载周期）44,190，而真正用于乘加的 PMU_MAC 只有 73,514。也就是说，CPU 花了和计算同量级的时间，只为了把权重搬进去。

于是把权重固化进片内 ROM：用 Block Memory Generator 生成一个 11066 字的单端口 ROM，内容由 Python 脚本从 C 头文件里的权重数组按 tap 顺序打包导出；每层启动前，软件只写一次 WBASE 指向该层起始字地址，加速器自行按 (w_base + oc × num_tap_groups + tap_group) 递推算地址。改造后 PMU_WLOAD 归零，axi-lite 写事务从 52,161 降到 7,068（剩下的全是输入、偏置与配置写），端到端周期从 1,489,638 降到 865,031（−41.9%）。

表 3　权重 ROM 地址规划（单位：32 位字）

层	起始字地址	字数	来源与说明
C1	0	42	6×5×5×1 打包，5 个 tap group × 6 通道
C3	42	608	16×5×5×6
C5	650	7680	120×4×4×16
F6	8330	2520	84×1×1×120
OUT	10850	210	10×1×1×84
Sobel	11060	6	2 个 3×3 核（Gx / Gy）

表中为 WBASE 地址划分,WBASE 寄存器用于配置当前网络层权重在片上 ROM 中的起始地址，为软件提供权重数据的寻址基址。

3.5 PMU 与 CPU 自定义 CSR

PMU 是本项目的基础设施，它有 6 个可读寄存器：PMU_MAC（处于 S_MAC 的周期数）、PMU_WLOAD（权重装载周期）、PMU_XLOAD（输入装载周期）、PMU_BUBBLE（S_MAC 中至少一个 lane 无效的周期数）、PMU_TOTAL（start 拍 + busy 周期的有效推理周期）、PMU_AXIWR（axi-lite 写事务数）。分别记录操作的周期数，PMU 通过 0x34 的 PMU_CTRL 使能并可写 1 清零，不改动任何计算逻辑，也不影响不读 PMU 的旧固件。

CPU 侧的 4 个自定义 CSR ：0xCC0 累计 MUL 指令数、0xCC1 累计 Load、0xCC2 累计 Store、0xCC3 累计流水线 Stall 周期。

PMU 用于记录加速器内部操作周期数，CSR 用于记录 E203 内部操作周期数，二者形成互补，是检测瓶颈，选择优化方向的基础。

3.6 NICE 紧耦合控制器

MMIO 启动一次推理需要三步：写 CTRL.start、轮询 STATUS.done、再写 CTRL=0 拉低 start。在 16 MHz 下，每次轮询都是一次跨总线的读，几百拍到上千拍不等。NICE 紧耦合是把启动与等待做成两条自定义指令，由核内直接与加速器握手，不经过总线。

实现上，cnn_nice_wrapper 解码 opcode = 0x7B（Custom3）的两条指令：funct3 = 1 为 cnn.start，funct3 = 2 为 cnn.wait。为了方便对照，CTRL[31] 保留了模式选择：默认走 NICE，写 CTRL 时自动切回 MMIO，这样同一份固件可以在两种模式下运行。

3.7 软件栈与 MNIST 应用闭环

软件栈分三层。最底层是寄存器头文件 cnn.h：把地址、位域与读写宏固定下来，并提供 cnn_layer() 与 cnn_pool() 两个内联函数把“配置—装载—启动—回读”封装成一次调用。中间层是 wbase_tap.h，定义每层的权重起始地址。最上层是 main.c：串起 6 层推理、argmax 与串口输出。

为了进行批量测试，上位机侧实现了串口批量协议：PC 发 RUN ，板子回 READY，PC 发 784 字节图像，板子回 cycles。并把每一张图的预测与周期数都落到 CSV，实现数据的批量记录。

固件里还内置了两组上板自检：2×2 max pooling 的 9 点自检（含负数窗口）与 Sobel 双核的 48 点非零自检。Sobel 不参与 LeNet 推理，而是借用同一套 3×3 MAC 通路完成 Gx/Gy 双核卷积，用来演示算子复用能力。两组自检把“回归”从 PC 搬到了板上，每次上电都能在串口直接看到 PASS/FAIL，详细结果见 5.3 节与附录 A.5/A.6。

第 4 章　优化方法与实验设计

在前文完成 E203 内核、ICB 总线桥、CNN 加速外设以及配套驱动软件栈的架构与接口设计基础上，本章介绍本设计遵循观测驱动迭代优化的完整技术路线，同时搭建对照基线、设计分层验证实验方案。整个优化迭代流程依托 CSR、PMU 硬件性能计数器采集运行信息，依次定位乘法运算瓶颈、权重传输瓶颈、算子实现瓶颈、同步轮询开销，分阶段实施软硬件协同改造；统一评价指标与测试约束，为第五章对最终系统性能、识别精度的量化分析提供可复现的实验基准。



图 4　系统迭代优化流程图

4.1 基于 CSR 与 PMU 的性能观测框架

本项目所有优化决策均以硬件计数器实测数据作为依据，搭建 CSR 与 PMU 双视角观测通路。RISC-V 内核侧 CSR 寄存器负责统计指令类型、Load/Store 指令数量、流水线 Stall 气泡周期；CNN 加速器内部集成 PMU 模块，用于统计 MAC 计算周期、数据读写事务、流水线有效与空闲周期。 网络推理遵循固定五步软件调度流程：配置 CFG0/CFG1 寄存器、写入 WBASE 权重基地址、载入激活数据、启动加速器并等待计算完成、顺序回读输出结果。我们在推理流程首尾打点采集计数器数值，实现逐层开销拆解，精准定位系统瓶颈。

4.1.1 第一轮观测：纯软件基线的乘法瓶颈

首先在 E203 内核上运行 LeNet 网络纯 C 推理版本，通过 CSR 统计指令分布。

表 4　CNN 加速器推理 Benchmark 性能统计结果

指标	数值	单位	说明与占比
乘累加指令数（mul）	281,640	条	占指令总数 33.09%
访存加载指令数（load）	563,563	条	占指令总数 66.22%，为当前主要开销项
访存存储指令数（store）	5,817	条	占指令总数 0.68%
指令总数（汇总）	851,020	条	mul + load + store
停顿 / 气泡周期数（stall）	282,786	cycle	占总周期 3.52%；有效执行周期 7,757,369 cycle（96.48%）
总周期数（cycles）	8,040,155	cycle	推理自启动至结束的完整时钟周期
CPU 主频	15,937,372	Hz	约 15.94 MHz
推理总时延	504	ms	校验：8,040,155 ÷ 15.937 MHz ≈ 504.5 ms

观测结果表明推理过程乘法指令占比很高，而 E203 原生内置乘法单元吞吐性能有限，乘法运算成为纯软件推理最主要的耗时来源。基于该结论，本设计采用经典 “RISC-V 处理器 + 专用硬件

加速器” 异构加速范式，将卷积中的密集乘累加运算卸载至 CNN 硬件加速器，CPU 仅负责网络层配置、调度与结果处理。

接入 CNN 加速器后再次使用 CSR 采集指令信息：

表 5　乘累加卸载至加速器后 Benchmark 性能统计结果

指标	数值	单位	说明与占比
乘累加指令数（mul）	829	条	占指令总数 0.27%，乘累加已卸载至加速器
访存加载指令数（load）	179,109	条	占指令总数 58.02%，以数据 / 权重读取为主
访存存储指令数（store）	128,768	条	占指令总数 41.71%，以向加速器写数据为主
指令总数（汇总）	308,706	条	mul + load + store
停顿 / 气泡周期数（stall）	866,896	cycle	占总周期 56.62%；有效执行周期 664,139 cycle（43.38%）
总周期数（cycles）	1,531,035	cycle	流水线自启动至结束的完整时钟周期
CPU 主频	15,936,716	Hz	约 15.94 MHz
推理总时延	96	ms	校验：1,531,035 ÷ 15.937 MHz ≈ 96.07 ms

对比 纯 C 基线与仅卷积硬件加速两组实测数据：在 E203 内核主频约 16 MHz 条件下，纯 C 版本总周期 8040155 周期，推理耗时 504 ms，乘法指令总量高达 282429 条，大量乘法运算与流水线阻塞是时延的核心来源。将卷积乘累加运算卸载至 CNN 加速器后，乘法指令数大幅下降至 829 条，乘法瓶颈得到显著缓解，整体推理时延降低至 96 ms。 但同时可以观察到新的性能问题：版本 Store 指令上涨至 128768 条，流水线 Stall 周期达到 866896 周期，Stall 占总周期比重很高。该现象源于权重数据需要 CPU 通过 MMIO 逐字搬运，在 ICB 转 axi-lite 单次单字写的总线机制下，频繁的写事务引发大量总线阻塞，计算瓶颈消失后，权重搬运的访存阻塞成为系统新的主要瓶颈。

4.1.2　第二轮观测：总线传输与权重搬运瓶颈

在乘累加计算卸载完成后，继续使用 CSR 观测 CPU 侧指令行为，发现系统中仍存在大量 Load、Store 指令，同时流水线 Stall 气泡周期占比较高，而气泡多由 load 和 store 指令产生。因此考虑将权重数据的传输通路由 axi-lite 总线 MMIO 写入方式调整为片上 ROM 预存储方案，借助 ROM IP 完成权重参数的固化与读取。

表 6　权重迁入片上 BRAM 后 Benchmark 性能统计结果

指标	数值	单位	说明与占比
乘累加指令数（mul）	10	条	占指令总数 0.02%，乘累加全部由加速器硬件完成
访存加载指令数（load）	24,968	条	占指令总数 57.12%，仅输入特征图读取，不再加载权重
访存存储指令数（store）	18,790	条	占指令总数 42.86%，向加速器/AXI 写中间结果
指令总数（汇总）	43,768	条	mul + load + store
停顿 / 气泡周期数（stall）	151,411	cycle	占总周期 19.33%；有效执行周期 632,005 cycle（80.67%）
总周期数（cycles）	783,416	cycle	流水线自启动至结束的完整时钟周期
CPU 主频	15,940,321	Hz	约 15.94 MHz
推理总时延	49	ms	校验：783416 ÷ 15.940 MHz ≈ 49.15 ms

由上表可以看出，将权重由 axi-lite MMIO 在线搬运改为预存至片上 BRAM、加速器通过 WBASE 寄存器直接寻址读取后，系统性能得到明显优化。对比单加速器版本，端到端总周期由 1531035 周期下降至 783416 周期，推理时延由 96 ms 降低至 49 ms。 从指令与硬件计数器统计结果来看：Store 指令由 128768 条降至 18790 条，流水线 Stall 周期由 866896 周期下降至 151411 周期，由权重逐字写入引发的总线阻塞开销被大幅削减。此时 CPU 侧乘法指令进一步下降至 10 条，计算侧压力基本释放。 配套 PMU 统计显示，加速器内部 MAC 有效计算周期为 73514 周期，流水线气泡 BUBBLE 为 4480 周期，MAC 阵列利用率较高；AXI 写事务仅剩余 7068 次，说明权重搬运带来的 AXI 写开销基本消除，系统瓶颈不再是权重传输，需要继续挖掘链路中剩余 Load、Store 与 Stall 的来源。

4.1.3 第三轮优化：算子通路复用拓展

权重搬运瓶颈解决之后，原有设计中池化、全连接运算仍然依靠 CPU 软件实现，仍会引入少量运算开销与额外指令。在不新增独立运算流水线、不改动 MAC 核心计算通路的前提下，利用 CFG 配置寄存器空闲比特位增加模式选择信号，对现有 CNN 计算通路进行复用：

1.2×2 最大池化：切换计算模式，将累加逻辑替换为取最大值逻辑；

2. 全连接层：将全连接运算等价映射为 1×1 卷积，复用 MAC 阵列完成计算。

表 7　增加多通道池化加速器后 Benchmark 性能统计结果

指标	数值	单位	说明与占比
乘累加指令数（mul）	10	条	占指令总数 0.02%，卷积乘累加由 CNN 加速器硬件完成
访存加载指令数（load）	22,733	条	占指令总数 52.30%，读取输入特征图数据
访存存储指令数（store）	20,737	条	占指令总数 47.68%，写回池化及卷积输出结果
指令总数（汇总）	43,480	条	mul + load + store
停顿 / 气泡周期数（stall）	195,026	cycle	占总周期 55.17%；有效执行周期 158,503 cycle（44.83%）
总周期数（cycles）	353,529	cycle	流水线自启动至结束的完整时钟周期
CPU 主频	15,937,372	Hz	约 15.94 MHz
推理总时延	22	ms	校验：353529 ÷ 15.937 MHz ≈ 22.18 ms

算子复用改造完成后，整个 LeNet 网络全部算子都由加速器硬件完成，消除剩余 CPU 侧软件算子开销，推理时延进一步降低至 22ms。虽然权重预存入 BRAM 后，权重传输带来的阻塞开销已大幅降低，但系统仍存在可观的流水线 Stall 周期。经 CSR 指令统计分析，该阻塞主要源于 E203 内核持续轮询加速器状态寄存器，反复读取状态以判断 CNN 计算任务是否完成。为消除轮询带来的 Load 访存开销与流水线停顿，本设计引入紧耦合自定义同步指令，替代循环轮询机制，以期进一步压缩端到端推理时延。

4.1.4 第四轮优化：紧耦合同步机制（NICE 指令扩展）

虽然权重预存入 BRAM 后，权重传输带来的阻塞开销已大幅降低，但系统仍存在可观的流水线 Stall 周期。经 CSR 指令统计分析，该阻塞主要源于 E203 内核持续轮询加速器状态寄存器，反复读取状态以判断 CNN 计算任务是否完成。为消除轮询带来的 Load 访存开销与流水线停顿，本设计引入紧耦合自定义同步指令，替代循环轮询机制，以期进一步压缩端到端推理时延。

因此我们设计 NICE 紧耦合同步方案，新增自定义指令替代轮询查询：加速器计算完成后通过硬件信号通知内核，无需 CPU 反复读取状态寄存器。

表 8　NICE 紧耦合同步优化后 Benchmark 性能统计结果

指标	数值	单位	说明与占比
乘累加指令数（mul）	10	条	占指令总数 0.03%，卷积乘累加由 CNN 加速器硬件完成
访存加载指令数（load）	14,901	条	占指令总数 41.83%，读取输入特征图数据
访存存储指令数（store）	20,723	条	占指令总数 58.14%，写回卷积、池化输出结果
指令总数（汇总）	35,634	条	mul + load + store
停顿 / 气泡周期数（stall）	222,802	cycle	占总周期 63.11%；有效执行周期 130,226 cycle（36.89%）
总周期数（cycles）	353,028	cycle	流水线自启动至结束的完整时钟周期
CPU 主频	15,939,338	Hz	约 15.94 MHz
推理总时延	22	ms	校验：353028 ÷ 15.939 MHz ≈ 22.15 ms
平均每条指令占用周期	9.91	cycle/条	总周期 ÷ 指令总数（衍生指标）
指令吞吐（IPC）	0.101	指令/cycle	指令总数 ÷ 总周期（衍生指标）

实测观测：CSR 统计显示轮询对应的 Load 指令数量明显减少， 但是端到端推理总时延几乎没有变化，仍然维持 22ms。 该现象说明：轮询 Load 指令带来的开销在整个链路占比极低，属于次要开销；虽然方案逻辑正确、能够减少轮询指令，但对整体性能收益微弱。因此该功能保留在工程代码中，作为可配置选项，但不作为核心创新性能收益。

本节以硬件性能计数器为观测手段，依次完成乘法算力、权重传输、算子实现、同步交互四个方向的迭代优化，通过多组对照 Benchmark 量化每一轮改造的收益，逐步收敛系统推理时延，同时验证了同步轮询开销对整体性能影响有限。

第 5 章　功能正确性验证

本章开展 CNN 加速器的功能正确性验证。硬件电路能够启动运行，不代表运算结果准确。数据通路、定点量化、控制逻辑及总线接口存在的细微错误会引入隐蔽数值误差，仅依靠最终分类结果难以识别。本文采用分层递进的验证策略，依次完成单元模块仿真、处理器 - 加速器全通路仿真、中间张量逐层数值比对与板级批量推理测试，形成自底向上的验证证据链。

5.1　仿真回归

CNN 内核的回归套件共 9 个用例：6 个卷积基础用例覆盖极小尺寸、stride=2、饱和边界、多输出通道、量化移位与边界条件，另有 2×2 max pooling、全连接 1×1 复用与 Sobel 双核各 1 个，一键脚本输出 PASS/FAIL，实测 9 PASS / 0 FAIL。其中原 6 个卷积用例在池化改动前后周期完全一致（22/22/22/128/226/22），说明 cfg_pool=0 的原路径零影响。完整用例与周期见表 10。

全通路仿真（CPU → ICB → axi-lite → cnn_regs → cnn_top）单独一套，用来确认桥与寄存器组能协同工作。这套仿真在加入池化改动后重新跑过一次并 PASS，确认 cfg_pool=0 的原路径未受影响。ROM 相关的仿真另作 xvlog/xelab 检查，0 错误。

表 9　CNN 内核仿真回归用例（9 PASS / 0 FAIL）

用例	形态/参数	周期	结果
t1_tiny	卷积 4×4×1, K=3	22	PASS
t2_stride2	stride=2	22	PASS
t3_saturate	全 127 饱和	22	PASS
t4_multi	C=2, OC=2	128	PASS
t5_requant	shift=4	226	PASS
t6_edge	边界/极值	22	PASS
t8_pool	2×2 max pooling	29	PASS
t9_fc	FC = 1×1 卷积	18	PASS
t10_sobel	K=3, OC=2	362	PASS

5.2　逐层数值比对

仅有“最终分类对”是不够的——分类对可能是巧合。因此在板子上跑完推理后，先用 riscv-nuclei-elf-nm 从 ELF 查出 7 个中间数组的 DTCM 地址，再用 OpenOCD dump_image 按字节回读，最后与 Python 黄金定点模型逐点比对。回读规模为 C1 3456 B、S2 864 B、C3 1024 B、S4 256 B、C5 120 B、F6 84 B、OUT 40 B。以 digit5 为例，七层全部 mismatch = 0、max_abs_diff = 0；权重预取修复与池化改动后这组比对均重新执行，结果不变。

这组比对还顺带证明了一件事：在 10 张样本里有 1 张（label=2）被误判为 4，而黄金模型给出的也是 4。也就是说，这个错例来自量化模型本身，而不是 RTL 或数据通路的错误。把“硬件错了”和“模型就这样”区分开，是逐层比对最大的价值。

5.3 板级自检与批量统计

每次上板都会自动运行两组自检并把结果打到串口：池化 9 点、Sobel 48 点。这是把“回归”从 PC 搬到板上的关键一步——9.1 节记录的 ROM 初始化路径故障，正是靠 Sobel 自检“上板全 0”暴露出来的。Sobel 在这里的角色是算子复用演示而非 LeNet 的验证项：它用与卷积完全相同的 3×3 MAC 通路、只换 Gx/Gy 两组权重，输出 6×6×2 的梯度图（图 5），方块内部两列/两行严格为 0，说明没有伪边缘。

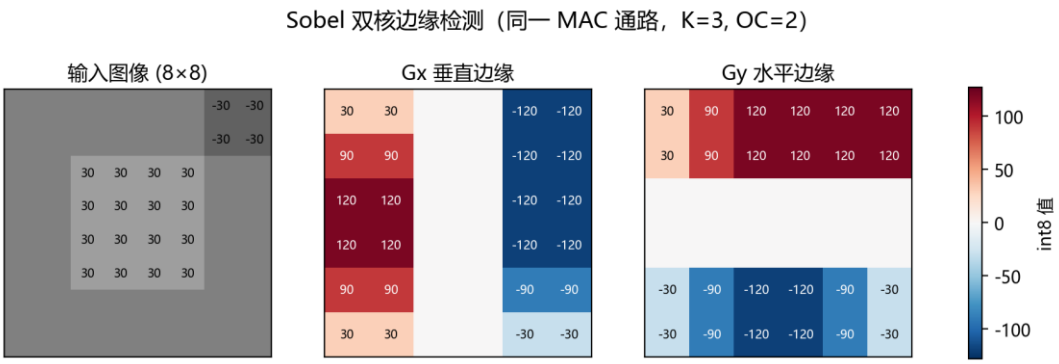


图 5　Sobel 双核边缘检测结果（同一 MAC 通路）

批量统计用 1000 张 MNIST 测试集跑完整链路，结果为 98.90%（989/1000），混淆矩阵见图 6。各类召回率在 95.5%～100% 之间，最低的是类别 8（95.5%）与类别 6（96.6%）；11 个错例中最多的是 6→0 与 8→9（各 2 例），其余为孤立错误。错例分布与 LeNet 这一类浅层网络的典型失效模式一致，没有出现“某一类系统性塌陷”这种硬件级故障特征。

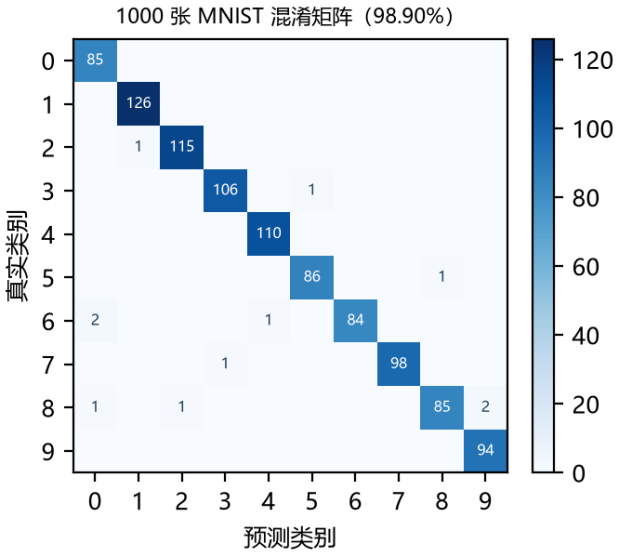


图 6　1000 张 MNIST 混淆矩阵

准确率的迭代提升过程同样构成一条验证证据链，如图 7。在 RTL 与硬件平台完全固定的条件下，仅修改软件侧的量化策略，推理准确率由 8% 逐步提升至 98%、甚至 100%。提升源自将全局固定移位量化替换为逐层标定移位，本方案最优 QUANT_SHIFT 为 9。

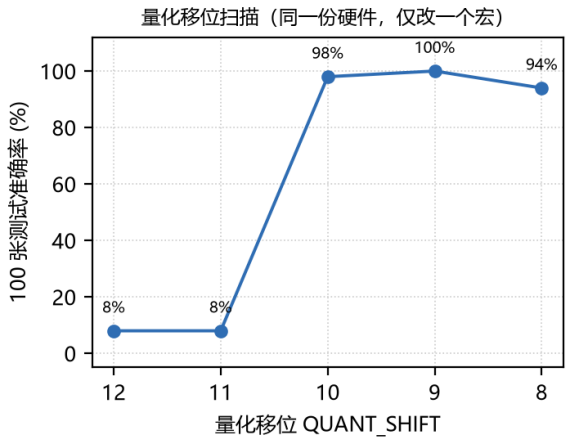


图 7　量化移位扫描与准确率提升路径

表 10　功能正确性验证矩阵

验证项	方法	结果	数据结果
E203 底座	CoreMark 上板运行 500 次迭代并校验 CRC	通过，2.118 CoreMark/MHz	附录 A.1
ICB → axi-lite 桥	寄存器读写比对 + 连续读写无死锁	通过	3.2 节
CNN 单层与多形态	9 个仿真用例一键回归	9 PASS / 0 FAIL	5.1 节
池化	仿真 t8_pool + 上板 9 点自检（含负数窗口）	9/9 一致	附录 A.5
全连接复用	t9_fc：矩阵形式与 1×1 卷积形式互比	结果完全一致（18 周期）	4.1.3 节
Sobel 双核（算子复用演示）	t10_sobel + 上板 48 点自检	48 非零点、内部为 0	图 5；附录 A.6
全通路	CPU→ICB→axi-lite→cnn_regs→cnn_top 仿真	通过	5.1 节
逐层数值	JTAG dump 7 个张量 vs 黄金定点模型	7 层 mismatch = 0	5.2 节
端到端准确率	1000 张 MNIST 测试集批量统计	98.90%（989/1000）	附录 A.4
连续稳定性	1000 张连续推理的周期极差	237 周期（14.8 μs）	6.4 节

第 6 章　性能与资源结果

6.1　测试条件

所有数据在同一套条件下取得：器件 XC7Z020-1CLG400C，内核时钟 16.000 MHz（实测主频 15,936,716 Hz，换算按 15.94 MHz），RISC-V GCC 10.1.0（-O2 及一组展开选项），MNIST 测试集前 1000 张，int8 输入，串口 115200 bps。

数据分三类，不可混用：（1）端到端周期取 main 中 t_end − t_start 的差值，起点为第一层配置之前，终点为最后一层输出回读之后；（2）PMU 为加速器内部计数，每次推理前写 PMU_CTRL 清零；（3）CSR 的 mul/load/store/stall 为运行前后取差值。三类数字在同一张表里出现时，均注明来源。

时延由周期数换算：t = cycles ÷ 15.94 MHz。

6.2　版本对比

表 12 与图 8 给出 6 个版本在同一口径下的端到端周期与加速比。从 V0 到 V5，端到端周期从 804 万降到 30.8 万，累计加速 26.08×；其中权重 ROM 常驻（V1→V2）贡献了 41.9%，硬件池化（V2→V3）贡献了约 55%（该区间同时包含一次层分块策略变更，故为上限估计，见注一），是两项收益最大的改动。

表 11　版本消融（同一口径：端到端周期）

版本	关键改动	端到端周期	耗时 (ms)	相对 V0 加速比
V0	E203 纯 C（六层全软件）	8,040,143	504	1.00×
V1	CNN 外设，权重经寄存器写入	1,531,035	96	5.25×
V2	权重预加载 ROM	783,416	49	10.26×
V3	＋多通道硬件池化	353,529	22	22.74×
V4	＋NICE 紧耦合启动/等待	353,028	22	22.77×
V5	最终版	308,284	19.27	26.08×

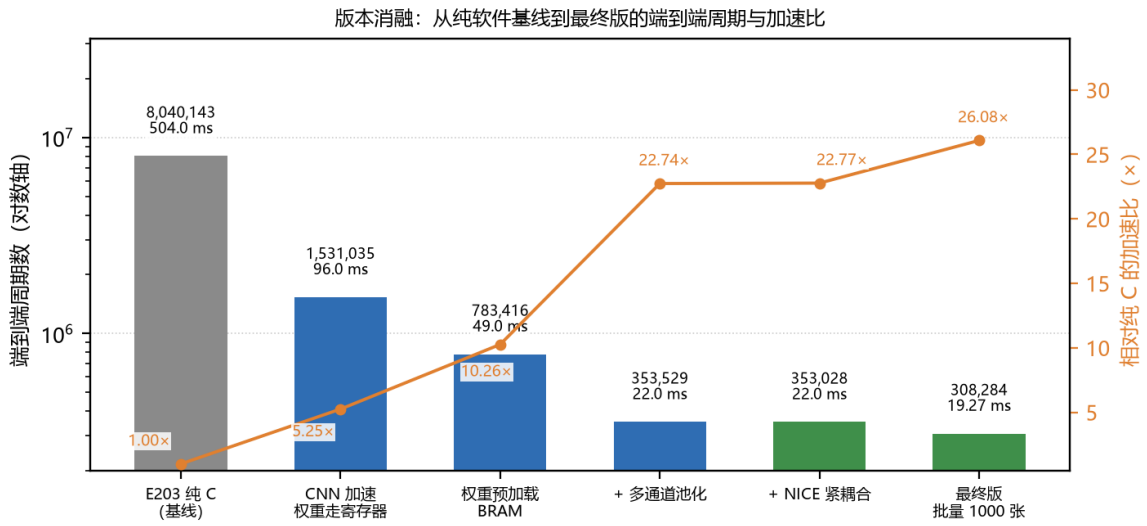


图 8　版本消融：端到端周期与加速比

6.3 PMU 计数

表 12 是 CNN 加速器的各类操作周期的采集统计，可借助 PMU 观测结果辅助验证本设计功能的正确性。

表 12 加速器侧 PMU 计数

计数项	计数值
PMU_WLOAD（权重装载周期）	74634
PMU_AXIWR（axi-lite 写事务）	12664
PMU_XLOAD（输入装载周期）	6588
PMU_MAC（MAC 阶段周期）	74634
PMU_BUBBLE（含无效 lane 的周期）	4480
PMU_TOTAL（有效推理周期）	86269

表 13 是 CPU 侧 CSR 的各类操作周期的采集统计

表 13 CPU 侧 CSR 计数（V0 / V1 / V2）

计数项	计数值
Load 指令数	14901
Store 指令数	20723
Stall 周期数	222802
MUL 指令数	10
端到端周期	353028
推理时间	22ms

结合表 12 与表 13 的 PMU 统计结果可以看出，E203 内核绝大多数乘法运算已成功卸载至 CNN 加速器的硬件 MAC 通路执行，再次印证了算子卸载方案的功能正确性；经过权重片上存储等一系列软硬件协同优化后，推理端到端时延降至 22 ms，直观体现出本优化方案具备良好的加速效果。

6.4 时延分布与确定性

端到端的可用性由分布决定，因此这里给出全部 1000 张样本的分布（图 9）。P50 = 19.268 ms、P90 = 19.271 ms、P95 = 19.272 ms、P99 = 19.273 ms、最大值 = 19.275 ms；持续吞吐约 51.9 张/秒。

值得注意的是极差：1000 张样本的最大值与最小值只相差 237 个周期（14.8 μs），占单张时延的 0.08%。具备优异的时序确定性和稳定性，能够满足端侧实时应用对时延抖动的约束要求。

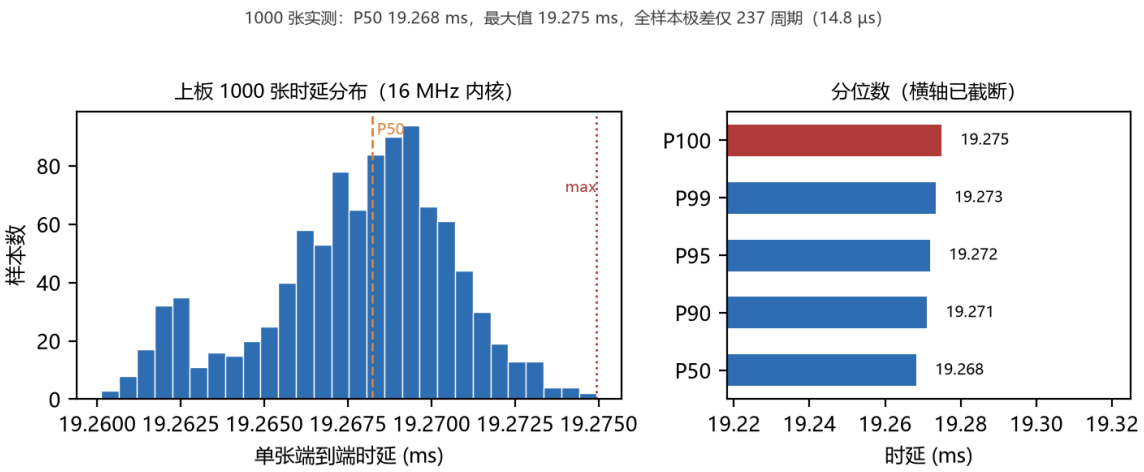


图 9　上板 1000 张时延分布与分位数

6.5　资源与时序

实现结果为：LUT 15,919（29.92%）、Slice Register 10,029（9.43%）、Block RAM 47.5 Tile（33.93%）、DSP48E1 11（5.00%），在 16 MHz（62.5 ns）约束下 WNS 0.000 ns、WHS 0.032 ns，50,081 个时序端点全部满足。设计在资源上有充足余量：LUT 仍剩 70%、DSP 仍剩 95%，说明这套架构可以继续加 lane 或加层而不会撞资源墙。

表 14　资源占用与时序（XC7Z020-1CLG400C）

资源	已用	可用	占比
LUT	15,919	53,200	29.92%
FF	10,006	106,400	9.43%
BRAMs	47.5	140	33.93%
DSP	11	220	5.00%
时序（16 MHz，62.5 ns）	WNS 0.000 ns / WHS 0.032 ns / TNS 0.000ns	50,081 端点	全部满足

第 7 章　大模型协作与技能包

7.1　三个关键 LLM 交互记录

五次协作里有三次值得展开，材料按当时的排查顺序整理。另外两次（JTAG 加载、ICB 桥字节掩码）只在表 16 里保留结论。

案例一：权重预取时序偏移

问题背景：C1 这类单通道层全对，C3/C5 多通道层从第 3 组 tap 起读错权重；单通道层完全正常，现象指向“多通道 + 同步读 ROM”这一组合。

Prompt（复述）：

现象：ROM 权重布局为 [OC][TAP_GROUP][LANE]，字地址 = w_base + oc × num_tap_groups + tap_group，w_rom 是同步读、输出比地址晚一拍。单通道层结果正确，多通道层从第 3 组 tap 起错。请分析同步读流水线与 tap 计数器的对齐关系，给出可验证的排查方案。

模型输出摘要：

判断 w_word 的更新与 tcur 的推进可能相差一个 tap-group 周期；· 建议对比 (tcnt+4)>>2 与 (tcnt+8)>>2 两种预取边界；· 建议逐拍打印 w_word_addr_r、tcur、tcnt，确认三者是否对齐。

人工决策与实施：

采纳“延迟与步长差一个 4”的判断； 在仿真里逐拍打印地址与 tap 编号，确认 tcnt=4 时应预取 base+2； 把 (tcnt+4)>>2 改为 (tcnt+8)>>2； 6 个卷积用例全过，随后 7 层逐层比对 mismatch=0。

模型把“单通道对、多通道错”翻译成流水线对齐问题，并给出了差 4 的假设。我们先用 C1/C3/C5 的对比把范围缩小，再设计逐拍打印实验确认实际差 8，改完后跑完 6 个用例和 7 层比对。模型提供了方向，具体差值靠波形读出。

案例二：ROM 初始化文件路径

问题背景：Sobel 仿真 PASS，上板 Gx/Gy 全 0；权重打包、输入装载、寄存器配置都已逐项排除。

Prompt（复述）：

现象：同一份 RTL，仿真用 TB 层次写 w_rom 通过，上板用 $readmemh 初始化后 Sobel 输出全 0。综合日志显示 $readmem data file 'rom_all_tap.hex' is read successfully，参数 W_TOTAL_WORDS=11066。请分析可能原因。

模型输出摘要：

方向一：检查权重打包顺序是否与 RTL 布局一致；· 方向二：检查 $readmemh 的文件是否更新；· 判据：日志里出现 read successfully 即可认为文件读取成功。

人工决策与实施：

先排除方向一，打包脚本和仿真一致，问题不在打包； 方向二部分成立，文件确实有新旧两份，但 read successfully 不能说明读取正确；真正判据为参数 W_TOTAL_WORDS=11066 和实际 ROM 深度 11060 不匹配，说明读的是旧的 11060 行文件； 把 rom_all_tap.hex 同步到 工程\rtl\cnn\ 后重新上板； Sobel 48 点与黄金模型一致。

模型提示“检查文件是否更新”，方向对。我们发现模型的判据不成立，从参数和深度的矛盾定位到具体文件，并查清了 Vivado 的搜索规则。这次模型给对了检查项，但给错了判定标准；日志里的 success 只能说明读到了同名文件。

案例三：CSR 累计口径

问题背景：同一固件多次上板，CSR 的 load/store/stall 一次比一次大，导致“优化前后对比”表失真。

Prompt（复述）：

现象：CSR 0xCC0–0xCC3 在多次加载 ELF 后持续累加。请分析计数器语义，确认是否存在清零通路，并给出软件侧可用的单次测量方案。

模型输出摘要：

指出 RTL 中 sel_mulcnt 只接了 csr_rd_en，没有写清零使能，csr_clear_*() 是空操作；· 建议改为“运行前后取差值”；· 提醒跨版本对比必须使用同一口径。

人工决策与实施：

核对 RTL，确认计数器只读无清零；修改 main.c，记录运行前基线，结束时取差；连续两次上板，差值完全一致（mul=10 / load=34114 / store=19260 / stall=202746）；撤回此前基于累计值的“优化前后对比”表。

模型从 RTL 里读出“只有读通路、没有清零”，并给出了差值方案。人工核对 RTL，改固件，连续两次上板确认差值一致，然后把此前基于累计值的对比表撤回。

表 15　大模型协作案例（提示词为事后复述，判断与实测为真实记录）

案例	问题类型	模型给出的方向	人工验证方式	结果
① JTAG 加载超时	调试链路	用 objcopy + load_image 绕过 CRC verify	改用 bin 路径后连续上板加载稳定	采纳
② ICB 桥字节掩码	总线实现	掩码使能未落到写数据路径	单字节/全字对照，定位到从机侧	部分采纳
③ 权重预取偏移	流水线对齐	延迟与步长差一个 4，核对 4/8 边界	逐拍打印地址与 tap 号；(tcnt+8)>>2 后 6 用例全过	采纳
④ ROM 文件路径	工具行为误解	方向对（查文件）但判据错（看 read successfully）	以 11066 参数 vs 11060 深度定案	部分采纳
⑤ CSR 累计口径	计数器语义	指出 CSR 只有读通路、无写清零	改差值口径后两次上板完全一致	采纳

7.2　技能包

表 16　技能包清单与归档状态

名称	适用场景	使用方法	失效条件	已验证效果	状态
踩坑清单（结构化）	任何 FPGA/SoC 联调项目	按“现象—触发条件—判据—处置”四段式检索	换器件或换工具版本后部分条目失效（如 $readmemh 搜索顺序）	docs/ 内 5 条，已入库	
ROM 一致性检查	权重固化到 ROM/IP 的方案	综合前运行，比对 4 份 ROM 副本与板级头文件，不一致退码 1	权重改由 CPU 写入时无意义	已拦住一次严重准确率事故，已入库	
逐层黄金比对	定点网络 + 有片上内存的加速器	导出黄金张量 → JTAG dump 硬件张量 → 自动 transpose 后逐点比对	改变了数据布局或量化规则后需同步更新	7 层 mismatch=0，已入库	
PMU 日志解析出表	带 PMU/CSR 计数的加速器	吃串口日志/JTAG dump/手工表格三种输入，自动出 CSV + 饼图 + 多版本对比图	计数口径变更后需同步修改派生指标定义	三种输入均用真实日志验证，已入库	
寄存器/地址头文件模板	CPU + 自研外设的 MMIO 方案	地址、位域、读写宏、层调用封装一次性生成	寄存器地址空间冲突时需重新规划	支撑 6 层推理与两次加餐，已入库	
TB 一键回归	任何带多个 testbench 的 RTL 工程	一条命令跑全部用例并输出 PASS/FAIL 摘要	用例依赖板级文件时会失效（本作品已刻意解耦）	7/7 PASS（10/10 含加餐）	待并入仓库

名称	适用场景	使用方法	失效条件	已验证效果	状态
串口批量测试	板上推理 + 上位机统计	RUN/READY/图像/PRED 四步协议，自动落 CSV 并统计准确率与时延	协议或串口参数变更后需同步两端	1000 张实测，当前在本地目录	待并入仓库
版本消融出表	多版本对比实验	从日志批量抽取同名指标，自动生成消融表与对比图	不同口径的数字混入会得出错误结论	本报告表 8/表 9 即由它产出	待并入仓库

7.3 复用验证与红线

技能包在两处做过迁移验证。ROM 一致性检查脚本从 A 侧复用到 B 侧：A 改完权重后跑一次，发现 IP 的 .mif 还是旧版本，省掉了一次上板全错再回头查的往返。PMU 日志解析脚本原来只吃串口日志，后来扩到串口、JTAG dump、手工表格三种输入，调用方代码一行没改。

团队定了四条不能破的规矩：模型输出不能直接当结论，必须有实测；未经人工阅读的代码不进主版本；估计值不能写成实测值；失败和回滚要留档，可以写进报告，但不能改写成成功。后两条在 4.5 节和第 9 章有具体例子。

第 8 章　复现与开源

8.1　仓库结构

当前仓库以 Vivado 2025.2 和 PYNQ-Z2（xc7z020clg400-1）为基线。仓库保存 RTL、ROM/IP 配置、构建脚本和验证证据；prj/ 目录以及综合、实现中间产物不纳入版本管理，复现时由基线工程重新生成。主要目录如下。

文件夹	介绍
board/pynqz2/	PYNQ-Z2 工程重建、比特流烧录 Tcl 脚本
ip/rom_all_tap/	Vivado 2025.2 的 ROM IP 配置、COE/MIF 与生成文件
rtl/	E203、CNN 加速器与 ICB-AXI Lite 桥 RTL
sdk/LeNet_oneP/	单张推理、Pool 与 Sobel 上板自检固件
sdk/LeNet_hw/	1000 张 MNIST 批量测试串口服务器
py/	权重导出、ROM 拼接与批量测试上位机脚本
python/golden/	黄金定点模型、逐层比对与 ROM 一致性检查
sim/	参数化 testbench 与一键仿真回归入口
vitis/	ARM 侧程序与接口工程
docs/reports/	技术报告、复现结果、原始日志与 CSV

8.2　工具版本说明

硬件复现使用 Vivado 2025.2，工程器件固定为 xc7z020clg400-1，顶层为 aic_rv32。ROM 由 Block Memory Generator 8.4 rev12 实现，数据宽度 32 bit、深度 16384。仓库中的 IP 配置和生成文件对应当前 PYNQ-Z2 基线；更换工程或目标板时仍需按新器件重新生成。

软件侧使用 Nuclei RISC-V GCC、hbird_sdk 0.1.4 和 GNU Make 编译 sdk/LeNet_oneP 与 sdk/LeNet_hw，JTAG 下载使用 OpenOCD 0.11 及 PYNQ-Z2 配置。批量上位机脚本依赖 Python 3、numpy 和 pyserial；PyTorch 与 torchvision 仅用于重新训练和导出黄金模型，不参与板端批量运行。

8.3　从零复现的步骤

图 10 给出复现链路，表 14 给出每一步的操作、命令和判定标准。复现前需准备一个已经加入 RTL、板级约束和 IP 的 PYNQ-Z2 Vivado 基线工程，并通过环境变量 E203_XPR 指定其 .xpr 路径；仓库不提交 prj/ 目录。ROM 权重更新后必须强制重新生成 IP，仅替换 COE 或 MIF 会继续使用旧 DCP。

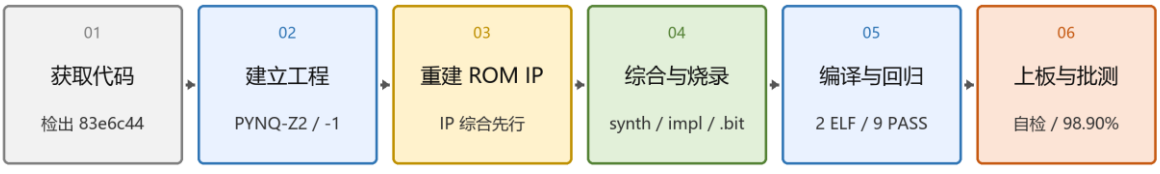


图 10　复现链路

表 17　PYNQ-Z2 基线复现步骤与判定

步骤	命令/操作	判定标准
1 获取代码	git clone https://github.com/NCabbage/e203_cnn.git git checkout 83e6c44	仓库根目录与 8.1 节一致
2 建立基线工程	Vivado 2025.2 选择 xc7z020clg400-1；加入 rtl/、板级 XDC 与 IP；确认顶层为 aic_rv32；设置 E203_XPR	器件和顶层正确，无缺失模块
3 检查 ROM 布局	python python/golden/assemble_rom_tap_sobel.py 核对 COE 与 MIF	11060 个 LeNet 字 + 6 个 Sobel 字，总计 11066
4 强制重建 ROM IP	vivado -mode batch -source board/pynqz2/repro_build.tcl 脚本先升级/对齐 IP，再执行 generate_target -force	rom_all_tap_synth_1 达到 100%，IP 不再 locked
5 综合与实现	同一脚本继续运行 synth_1 和 impl_1；实现指令为 Explore	综合 0 errors；WNS=0.000 ns，WHS=-0.025547 ns
6 生成并烧录比特流	impl_1 运行至 write_bitstream；运行 program_bit.tcl	生成 aic_rv32.bit，烧录后 startup status=HIGH
7 编译软件	分别编译 sdk/LeNet_oneP 与 sdk/LeNet_hw（必须强制重编）	生成两个 ELF，编译 0 errors
8 仿真回归	powershell -ExecutionPolicy Bypass -File sim/run/run_all.ps1	9 PASS / 0 FAIL
9 上板自检	OpenOCD 下载 LeNet_oneP.elf；COM9，115200 8N1	PRED=3；Pool 9/9 PASS；Sobel 48 点 PASS
10 批量测试	OpenOCD 下载 LeNet_hw.elf；运行 py/uart_batch_mnist.py --port COM9	LeNet_hw 默认 57600；989/1000，准确率 98.90%，生成 CSV

判定时应在综合前检查 ROM IP 的 PART 与版本，并确认 rom_all_tap_synth_1 已完成；若这两项未满足，不应继续顶层综合。本轮实现存在唯一时序残留 WHS=-0.025547 ns，功能与批测已通过，但严格“时序无违例”标准仍未满足报告原表第 3 步的要求。

8.4　一键回归与开源声明

仿真一键回归入口为 sim/run/run_all.ps1。脚本统一执行编译、精化和 9 个用例，末尾打印 PASS/FAIL、总计数和日志目录；任一用例失败时返回非零退出码。当前 PYNQ-Z2 基线结果为 9 PASS / 0 FAIL。

板端自检使用 LeNet_oneP，串口为 115200；批量测试使用 LeNet_hw 和 py/uart_batch_mnist.py，串口默认 57600。批量协议采用 BEG/DATA/END，每 128 字节返回 ACK，主机超时重传。原始 CSV、串口日志、时序摘要和复现说明保存在 docs/reports/2026-10-30_第八章复现验证/。

开源方面，本作品整体采用 Apache-2.0 许可，与 E203 保持一致。处理器底座来自蜂鸟 E203；CNN 内核、PMU、NICE wrapper、CSR、仿真平台和上位机脚本由本队实现，量化权重由训练脚本导出。

第 9 章　展望

本文完成了面向 INT8 卷积推理的软硬件协同加速方案设计与验证，在时延确定性、推理吞吐上取得了预期效果，但当前方案仍存在可拓展与优化空间，后续可从以下方向继续完善。

9.1　优化方向

从本设计最终的 CSR 性能统计结果可以看出，系统仍存在大量访存加载、存储指令以及流水线停顿周期，E203 内核与 CNN 加速器之间的数据交互通路是当前最主要的性能瓶颈。受项目开

发周期约束，部分预设的优化方案未能在本次工作中落地，后续可以从短期通路优化与长远处理器架构迭代两个方向继续完善。

短期优化可围绕数据传输机制开展：采用 32bit/64bit 位宽打包传输、引入 AXI 突发传输机制，降低交互带来的停顿开销；也可对 E203 处理器本身进行优化，改进乘法单元运算效率、增加分支预测模块，以此削减流水线气泡与 stall 周期。

从长远角度来看，可自主设计多级流水线处理器内核，通过加深流水线级数提升处理器主频与运算吞吐，进一步提升整套加速系统的推理性能。

9.2 拓展方向

除上述性能优化思路外，本系统还可开展功能层面的拓展开发。当前设计以离线载入数据集样本完成推理验证，后续可外接摄像头模块，实现实时图像采集与预处理；同时增加 HDMI 显示输出通路，将识别结果、标识信息推送至显示设备，搭建“图像采集 - CNN 推理 - 结果可视化”的完整实时视觉应用链路，进一步丰富演示形态，贴合真实端侧视觉应用场景。

9.3 后续工作

基于本课题完成的软硬件协同 CNN 加速原型的测试与分析，本方案验证了紧耦合加速架构在低抖动端侧 INT8 推理场景的可行性。结合实测性能瓶颈与应用场景需求，后续改进计划分为性能优化与功能拓展两大方向，分别在 9.1、9.2 中展开详述。性能优化聚焦内核与加速器之间的数据交互瓶颈，旨在降低传输停顿、提升推理吞吐；功能拓展面向真实视觉落地场景，完善采集、交互与可视化链路，丰富系统的工程演示能力。

附录 A　原始日志摘录

本附录摘录关键原始日志，便于评委核对正文数字。为控制篇幅，仅保留与结论直接相关的行；完整日志在仓库 report/ 与板端工程中。

A.1　CoreMark（E203 底座）

```
HummingBird SDK Build Time: Sep 23 2026, 09:10:42
CPU Frequency 15936716 Hz
CoreMark Size    : 666
Total ticks      : 236054487
Total time (secs): 14.812167
Iterations/Sec   : 33.756034
Iterations       : 500
Compiler version : GCC10.1.0
Memory location  : STACK
[0]crclist : 0xe714  [0]crcmatrix : 0x1fd7  [0]crcstate : 0x8e3a  [0]crcfinal : 0xa14c
Correct operation validated.
(Iterations*1000000/total_ticks) = 2.118155 CoreMark/MHz
```

A.2　纯 C 基线（V0）

```
===== Benchmark Result =====
Total cycles: 8040143
Total mul num: 282429
Total load num: 639234
Total store num: 66842
Total stall num: 499400
CPU Frequency: 15940321 Hz
Total time: 504 ms
```

A.3　加速器版本（V1 寄存器权重 / V2 ROM 权重）

```
--- V1 权重经寄存器写入 ---
Total cycles: 1531035
Total mul num: 829  Total load num: 179109
Total store num: 128768  Total stall num: 866896
Total time: 96 ms
--- V2 权重预加载 ROM ---
Total cycles: 783416
PMU MAC=73514 WLOAD=73514 XLOAD=2108 BUBBLE=4480 TOTAL=82907 AXIWR=7068
Total mul num: 10  Total load num: 24968
Total store num: 18790  Total stall num: 151411
Total time: 49 ms
```

A.4　最终版批量测试（V5，1000 张）

```
#define QUANT_SHIFT   9
===== Batch Result =====
Total:      1000
Correct:    989
Accuracy:   98.90%
Avg cycles: 308284

#define QUANT_SHIFT   8
Total: 1000   Correct: 913   Accuracy: 91.30%   Avg cycles: 307856

# 100 张扫描（QUANT_SHIFT = 12/11/10/9/8）
Accuracy:   8.00% / 8.00% / 98.00% / 100.00% / 94.00%
```

A.5　池化上板自检（含负数窗口）

```
===== Pooling Selftest (2x2 max) =====
  pool[0] = -2  (expect -2)
  pool[1] = 10  (expect 10)   pool[2] = 11  (expect 11)
  pool[3] = 19  (expect 19)   pool[4] = 22  (expect 22)
```

```
  pool[5] = 24  (expect 24)   pool[6] = 31  (expect 31)
  pool[7] = 34  (expect 34)   pool[8] = 36  (expect 36)
POOL SELFTEST: PASS
```

A.6 Sobel 上板自检（K=3, OC=2）

```
  Gx (vertical edges, 6x6)          Gy (horizontal edges, 6x6)
    30   30    0    0 -120 -120       30   90  120  120  120  120
    90   90    0    0 -120 -120       30   90  120  120  120  120
   120  120    0    0 -120 -120        0    0    0    0    0    0
   120  120    0    0 -120 -120        0    0    0    0    0    0
    90   90    0    0  -90  -90      -30  -90 -120 -120  -90  -30
    30   30    0    0  -30  -30      -30  -90 -120 -120  -90  -30
  nonzero = 48 (expect 48)  Gx-mid-cols-zero = 1  Gy-mid-rows-zero = 1
SOBEL SELFTEST: PASS
```

附录 B　关键文件清单

表 18　关键文件清单

路径	作用
rtl/cnn/cnn_top.v	INT8 CNN 内核：4-lane MAC、ROM 接口、池化、状态机
rtl/cnn/cnn_regs.v	寄存器组：配置、指针、PMU、WBASE、NICE 模式选择
rtl/cnn/cnn_quant.v	量化 + ReLU + int8 饱和
rtl/cnn/PMU_REGISTERS.md	PMU 寄存器定义、用法与验证摘要
rtl/subsys/cnn_nice_wrapper.v	NICE 紧耦合控制：cnn.start / cnn.wait 两条自定义指令
rtl/axi_lite_slave/	ICB → axi-lite 协议桥
rtl/core/e203_exu_csr.v	CPU 侧自定义 CSR（0xCC0–0xCC3）计数实现
LeNet_oneP/include/cnn.h	寄存器地址、位域、读写宏与层调用封装
LeNet_oneP/include/wbase_tap.h	每层权重在 ROM 中的起始字地址
LeNet_oneP/application/main.c	MNIST 应用闭环、PMU 打印、池化与 Sobel 上板自检
python/golden/	黄金定点模型、逐层比对、ROM 一致性检查、权重打包导出
python/pmu/pmu_report.py	PMU/CSR 日志解析与出表出图
docs/reports/	四份技术报告：PMU 解析与 BRAM 对比、池化、FC 复用与 Sobel、82% 根因
report/build/	本报告与全部图表的生成脚本（数据 → 图 → 文档，可重跑）

参考文献

[1] 胡振波. 手把手教你设计 CPU：RISC-V 处理器篇[M]. 北京: 人民邮电出版社, 2018.

[2] 胡振波. RISC-V 架构与嵌入式开发快速入门[M]. 北京: 人民邮电出版社, 2019.

[3] SI-RISCV. e200_opensource（蜂鸟 E203 开源 RISC-V 处理器）[CP/OL]. Apache-2.0, https://github.com/riscv-mcu/e203_hbirdv2.

[4] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.

[5] Jacob B, Kligys S, Chen B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[C]// CVPR. 2018: 2704-2713.

[6] AMD/Xilinx. Vivado Design Suite User Guide: Synthesis (UG901)[Z]. 2019.

[7] AMD/Xilinx. Vivado Design Suite User Guide: Implementation (UG904)[Z]. 2019.

[8] AMD/Xilinx. UltraFast Design Methodology Guide for the Vivado Design Suite (UG949)[Z]. 2019.